Solution Document: AWS Environment Setup & Infrastructure Automation — Student Record & Academic Management Platform

Name: Om Bhosale | **PRN:** 2125UDSM1037 | **Date:** 16 October 2026

This document records the AWS environment setup and infrastructure automation completed for the Student Record & Academic Management Platform. It follows the trainer's Chapter 2 lab guide (Enterprise Smart Library Platform) and extends it with the Terraform and Ansible work that came next. Each step is backed by a screenshot; AWS keys, the account ID and IP addresses are blurred.

1. Scope and environment

Chapter 2 requires three tools, programmatic credentials and an SSH key pair before any infrastructure is built. Every Library-specific name in the lab guide was replaced with a Student Record equivalent.

Item	Lab guide (Library platform)	This implementation (Student Record platform)
Workstation	Ubuntu 22.04 / macOS / WSL2	WSL2 Ubuntu 26.04 on Windows
AWS CLI v2	2.15.x or higher	2.38.1
Terraform	1.7.x or higher	1.16.6 (linux_amd64)
Ansible core	2.16.x or higher	2.21.5
IAM user	devops-trainer-admin (example)	devops-student
SSH key pair	library-aws-key	student-aws-key
AWS region	us-east-1	ap-south-1 (Mumbai) for deployment
Project folder	n/a	~/student-platform (terraform/ and ansible/)

2. AWS IAM user for programmatic access

Terraform and the AWS CLI cannot run without credentials, so an IAM user named devops-student was created in the AWS console (Figure 1). It must be allowed to manage the services the Terraform code uses: EC2, VPC, Elastic IP and S3. The user's access key was entered only through `aws configure` and is never written into project files.

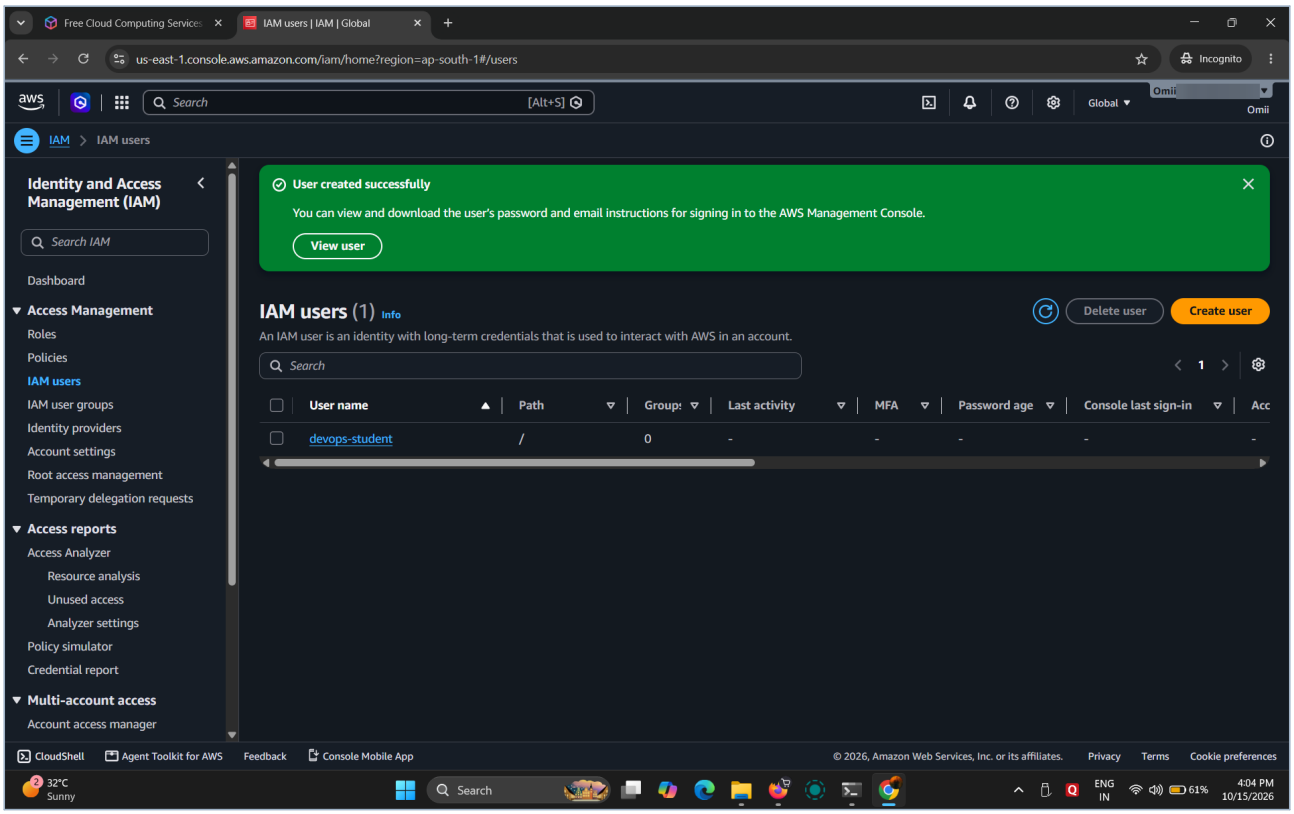


Figure 1: IAM console confirming that the user devops-student was created successfully (account ID blurred)

3. Section 2.1: Installing AWS CLI, Terraform and Ansible

All four steps of the lab guide were run in the WSL2 Ubuntu terminal. Every tool was already at its newest version, so the commands confirmed the installation rather than changing it.

3.1 Step 1: Workstation baseline utilities

The package index was refreshed and the baseline packages (curl, wget, unzip, jq, git, gnupg, Python 3 and related tools) were checked. The index fetch downloaded 3,524 kB and every package reported as already the newest version.

```
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y curl wget unzip jq git software-properties-common \
  ca-certificates gnupg apt-transport-https python3 python3-pip python3-venv lsb-release
```

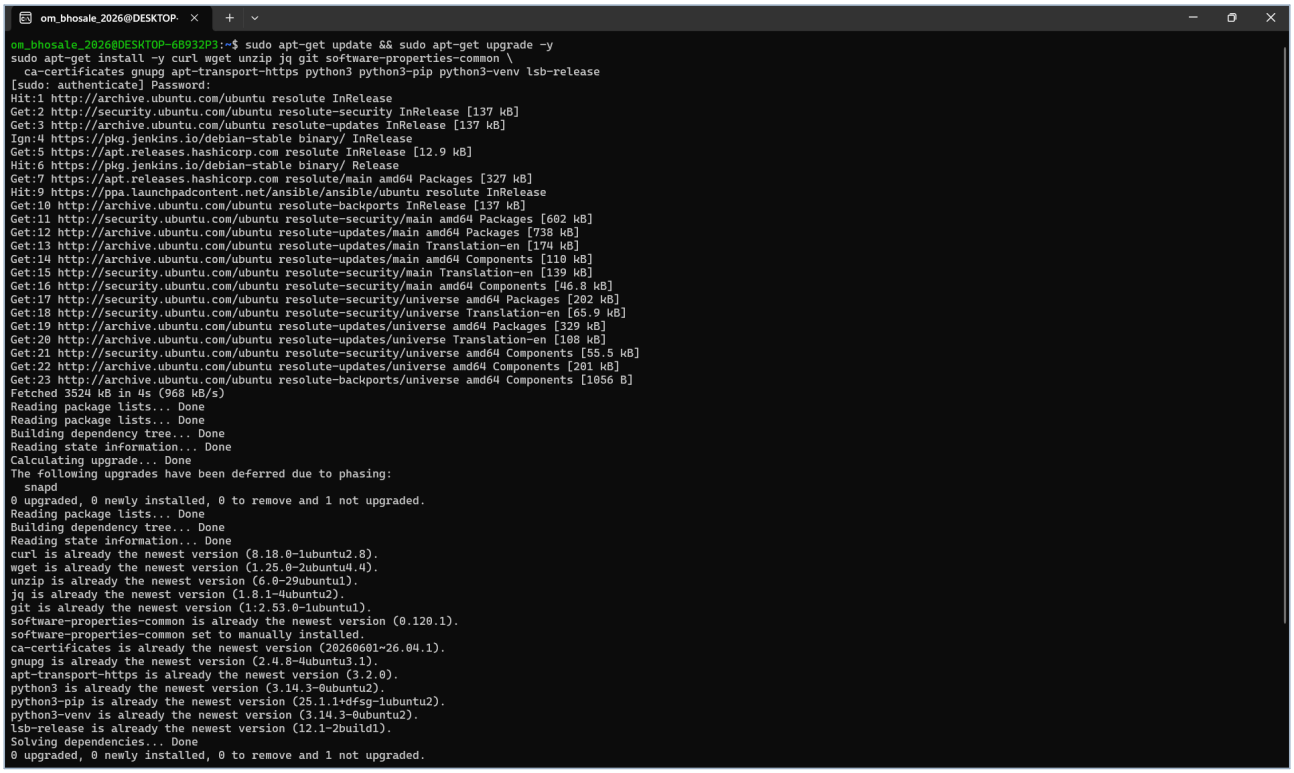


Figure 2: apt-get update, upgrade and baseline tool installation

3.2 Step 2: AWS CLI v2

The official v2 bundle was downloaded (70.69 MB), unzipped and passed to the installer. The installer found the same version already present and skipped it. `aws --version` reports aws-cli/2.38.1, above the required 2.15.x.

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip -q awscliv2.zip
sudo ./aws/install --update
rm -rf aws awscliv2.zip
aws --version
```

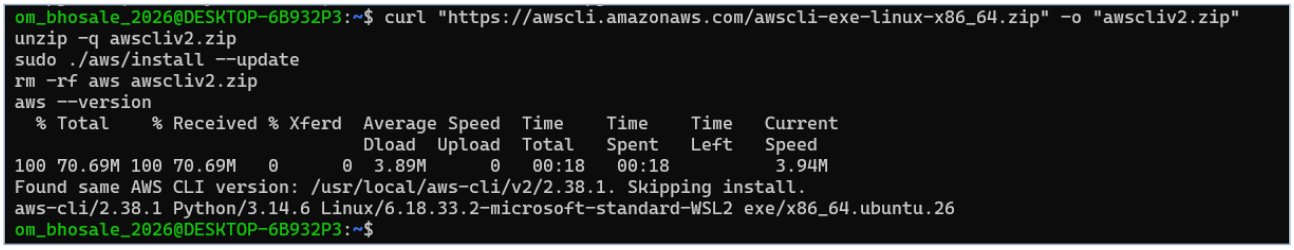


Figure 3: AWS CLI v2 download and version check (aws-cli/2.38.1)

3.3 Step 3: HashiCorp Terraform

The HashiCorp repository was registered and Terraform installed from it. `terraform -version` reports v1.16.6 on linux_amd64, above the required 1.7. Two messages in the output are harmless: gpg declined to overwrite the keyring file from an earlier install (the repository still verified, as the later `apt-get update` shows), and the autocomplete installer reported that it was already present in `~/.bashrc`.

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_rele
sudo apt-get update
sudo apt-get install -y terraform
terraform -install-autocomplete
terraform -version
```

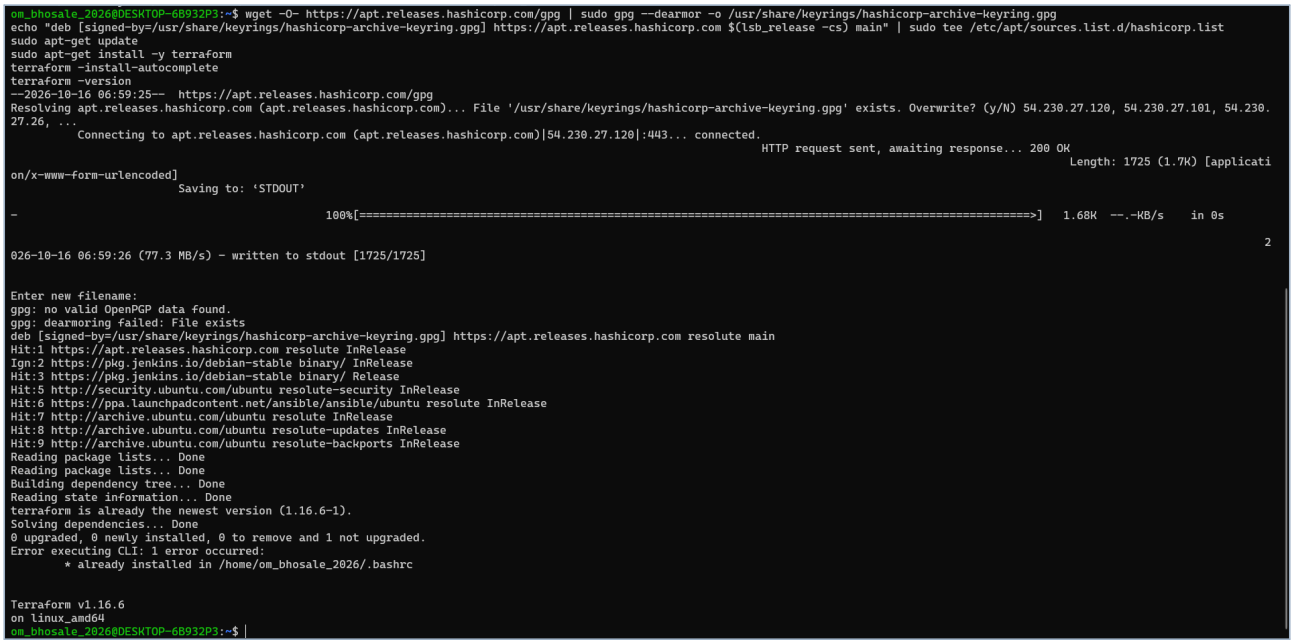


Figure 4: HashiCorp repository setup and Terraform v1.16.6

3.4 Step 4: Ansible core and collections

The Ansible PPA was added, Ansible installed (package 14.5.0) and the amazon.aws and community.general collections installed. `ansible --version` reports ansible [core 2.21.5] running on Python 3.14.4, above the required 2.16 and Python 3.10.

```
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt-get install -y ansible
ansible-galaxy collection install amazon.aws community.general
ansible --version
```



Figure 5: Ansible PPA, installation, collections and version check

4. Section 2.2: AWS authentication and STS verification

`aws configure` stored the access key, secret key, default region (ap-south-1) and output format (json) under `~/.aws/`. `aws sts get-caller-identity` then returned the authenticated principal, and its ARN ends in `user/devops-student`, which proves the credentials work. The key values, UserId and account ID are blurred in Figure 6.

```
aws configure
```

```
aws sts get-caller-identity
```



Figure 6: aws configure and STS identity check (keys, UserId and account ID blurred)

The CLI warned that `~/.aws/credentials` is readable by other users. The lab guide's governance note says credentials must stay out of Git and `.tf` files; restricting the file is a one-line fix:

```
chmod 600 ~/.aws/credentials
```

5. Section 2.3: SSH key pair generation and EC2 import

Ansible reaches the server over OpenSSH, so a 4096-bit RSA key pair named student-aws-key is used. The pair already existed from the previous day, so ssh-keygen asked to overwrite it and the write failed with Permission denied because the private key is read-only (mode 400). That is a safe outcome: the existing pair was kept, the permissions were reapplied (400 private, 644 public) and `ls` confirms them. The public key was then imported, and AWS returned KeyName student-aws-key with KeyPairId key-0f17da4b5963fae68.

```
mkdir -p ~/.ssh
ssh-keygen -t rsa -b 4096 -f ~/.ssh/student-aws-key -N "" -C "devops@student-platform"
chmod 400 ~/.ssh/student-aws-key
chmod 644 ~/.ssh/student-aws-key.pub
ls -la ~/.ssh/student-aws-key*
aws ec2 import-key-pair --key-name "student-aws-key" \
  --public-key-material fileb://~/.ssh/student-aws-key.pub --region us-east-1
```



Figure 7: SSH key generation, permissions and EC2 key pair import

A key pair is regional. The lab guide imports it into us-east-1, but Terraform launches the instance in ap-south-1, so the same public key must also be registered in ap-south-1 (same command with `--region ap-south-1`) for `key_name = "student-aws-key"` to resolve there.

6. Section 2.4: Toolchain verification matrix

All five checks from the lab guide's matrix passed, and the observed values meet or exceed every minimum.

Tool	Verification command	Expected	Observed	Evidence	Status
AWS CLI v2	`aws --version`	aws-cli/2.15.x or higher	aws-cli/2.38.1	Figure 3	Verified
AWS authentication	`aws sts get-caller-identity`	Valid account ID and IAM user ARN	ARN ending user/devops-student	Figure 6	Verified
Terraform CLI	`terraform -version`	v1.7.x or higher	v1.16.6 on linux_amd64	Figure 4	Verified
Ansible core	`ansible --version`	core 2.16.x or higher	core 2.21.5	Figure 5	Verified
SSH key pair	`aws ec2 import-key-pair` response	student-aws-key present in AWS	KeyName student-aws-key, KeyPairId key-0f17da4b5963fae68	Figure 7	Verified

7. Terraform: provisioning the AWS infrastructure

With the toolchain verified, Terraform builds the cloud layer: network, security group, compute host and backup bucket. The code lives in `~/student-platform/terraform`. Each file was created with `cat > file <<'EOF'`, because code is saved to a file first and never pasted at the shell prompt. The optional S3 remote-state backend is left commented out, since its bucket does not exist on a first run.

7.1 Configuration files

File	What it defines
providers.tf	Terraform 1.7.0 or higher, AWS provider ~> 5.0, region taken from a variable
variables.tf	aws_region ap-south-1, VPC 10.1.0.0/16, public subnet 10.1.1.0/24, t3.micro, key_name student-aws-key, app_port 8080
network.tf	VPC with DNS support, internet gateway, public subnet in ap-south-1a, route table sending 0.0.0.0/0 to the gateway, subnet association
security.tf	Security group: SSH (22) from one admin IP /32 only, HTTP (80) from anywhere, all outbound allowed
s3.tf	Backup bucket named student-platform-backups plus the account ID, with all four public-access blocks enabled
ec2.tf	Latest Ubuntu 22.04 AMI from Canonical, EC2 instance with a 20 GB encrypted gp3 root volume, and an Elastic IP
outputs.tf	app_server_public_ip (for the Ansible inventory) and s3_backup_bucket

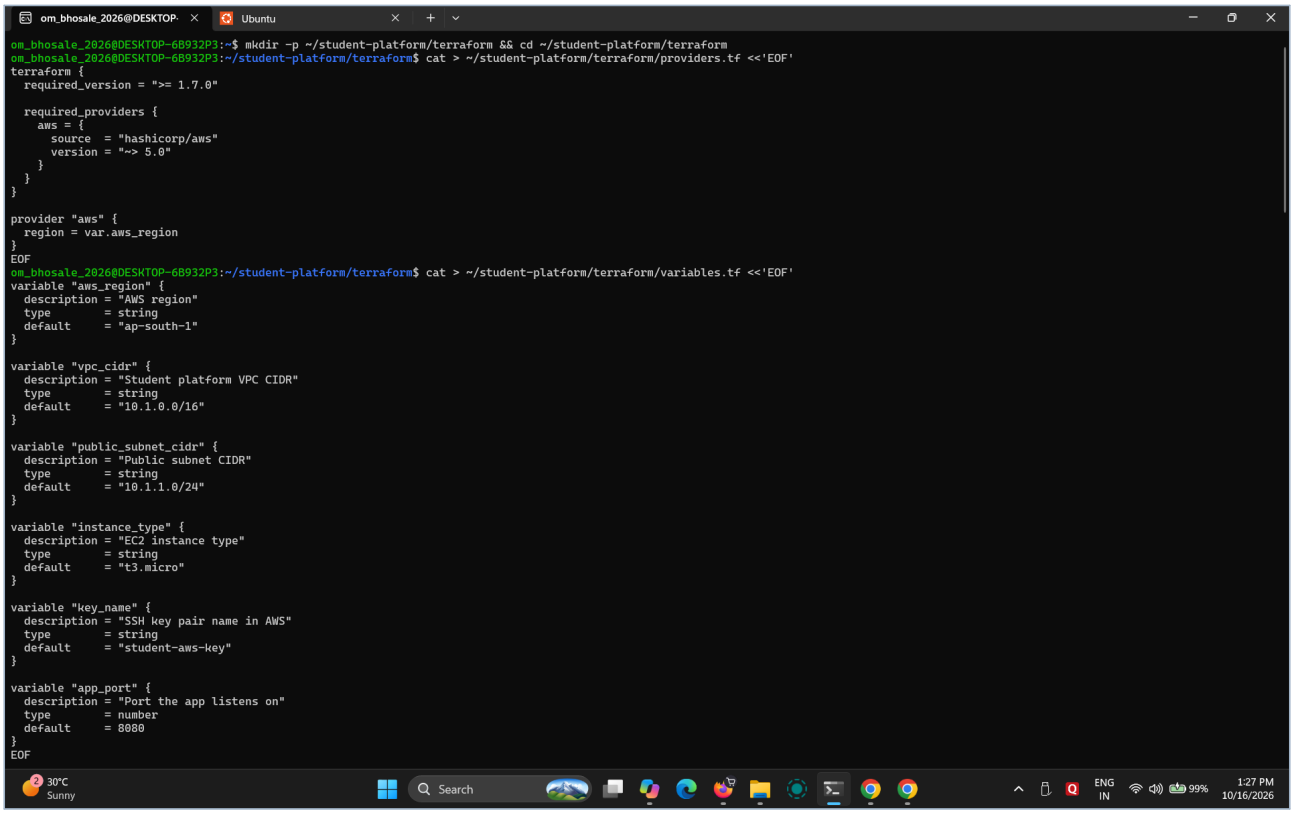


Figure 8: providers.tf and variables.tf

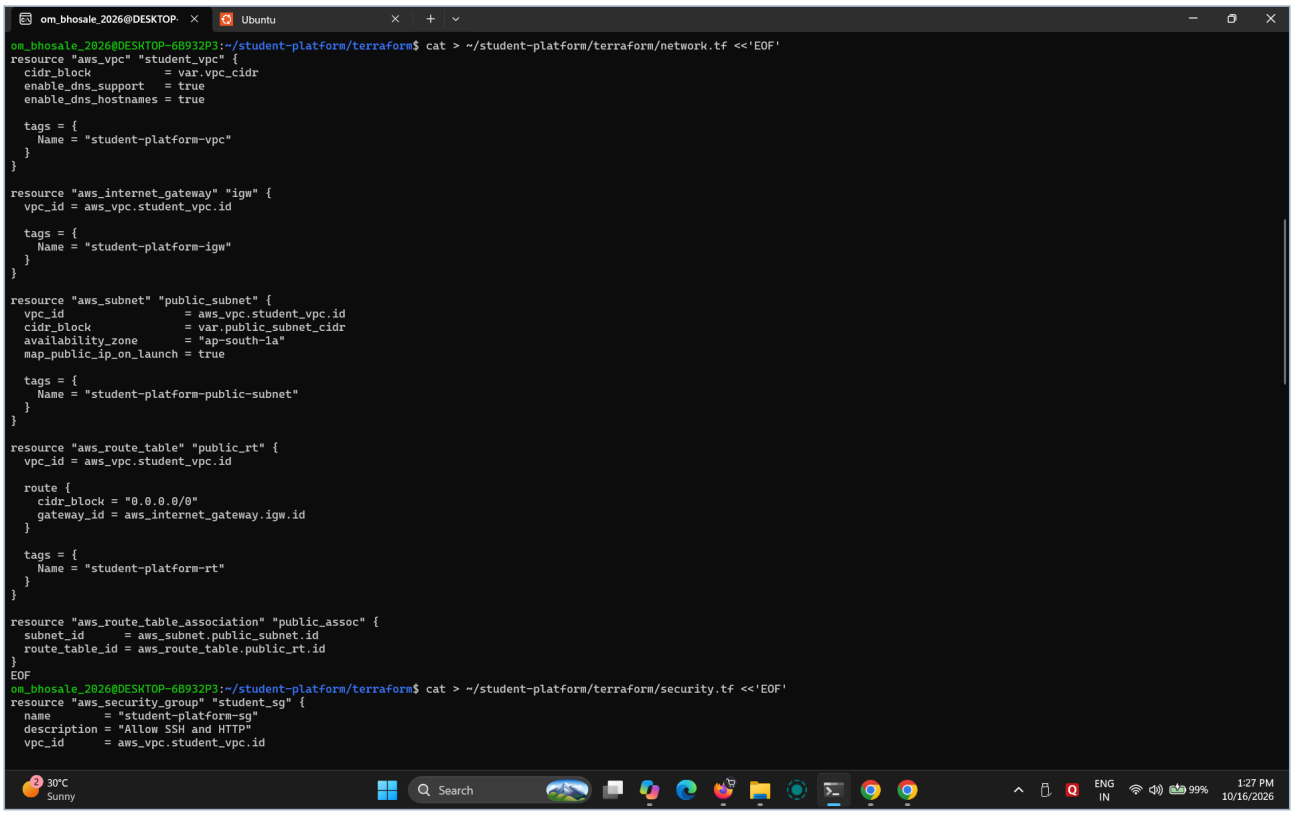


Figure 9: network.tf (VPC, gateway, subnet, route table) and the start of security.tf

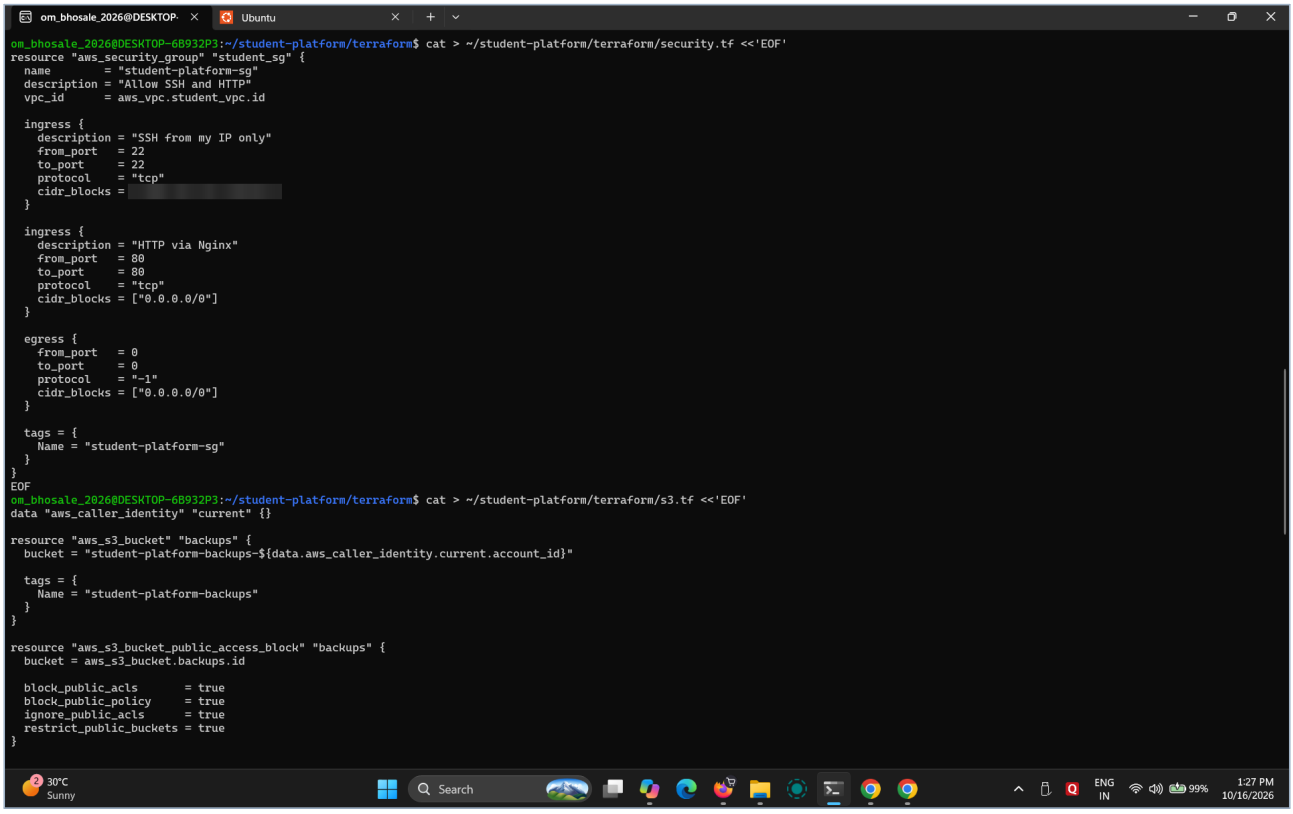


Figure 10: security.tf and s3.tf (the SSH source IP is blurred)

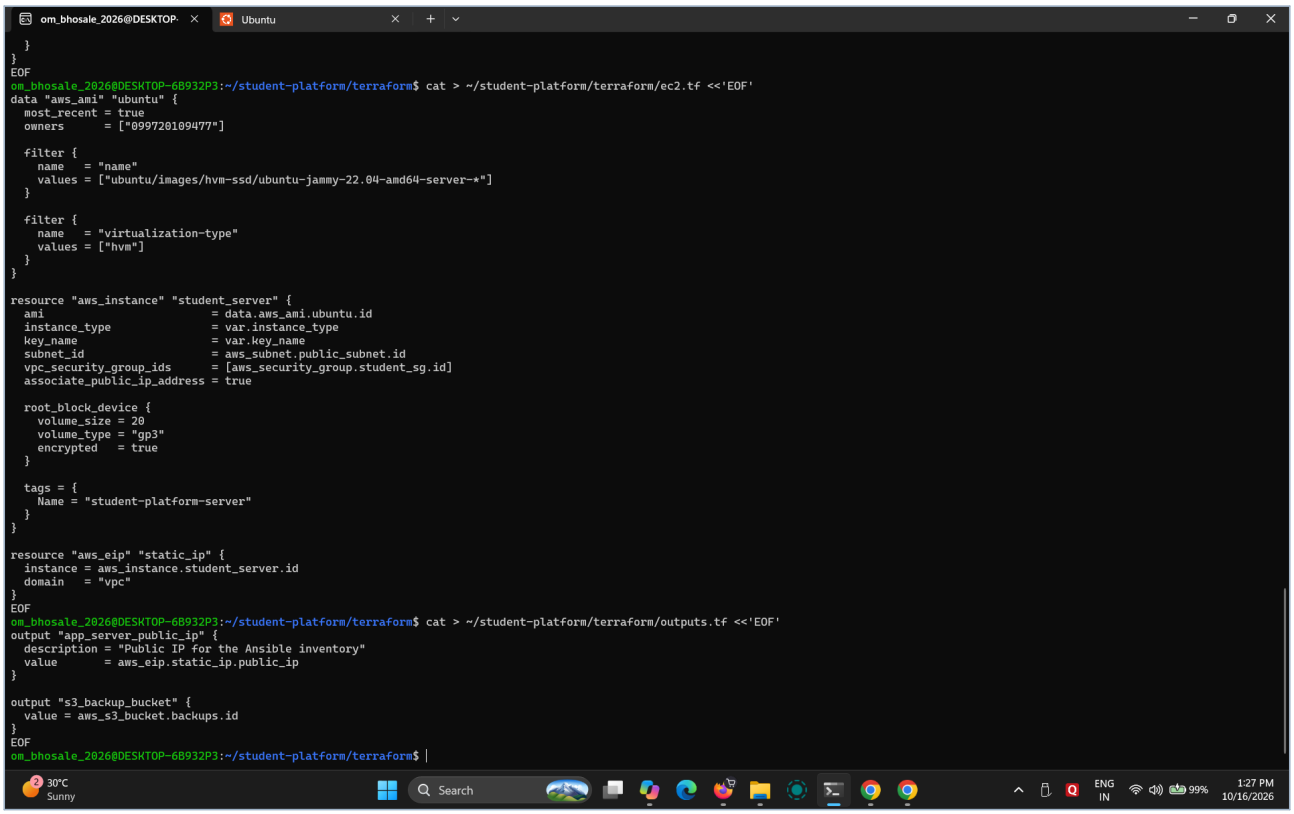


Figure 11: ec2.tf and outputs.tf

SSH is limited to a single administrator address, the bucket blocks all public access and the root disk is encrypted, so the infrastructure follows the lab guide's security governance note.

7.2 init, validate, plan and apply

`terraform init` installed hashicorp/aws v5.100.0 (signed by HashiCorp) and wrote the lock file `.terraform.lock.hcl`. `terraform validate` returned "Success! The configuration is valid." `terraform plan` read the caller identity and the Ubuntu AMI, then reported 12 resources to add, 0 to change and 0 to destroy, with `app_server_public_ip` and `s3_backup_bucket` known only after apply.

```
terraform init
terraform validate
terraform plan
terraform apply
```

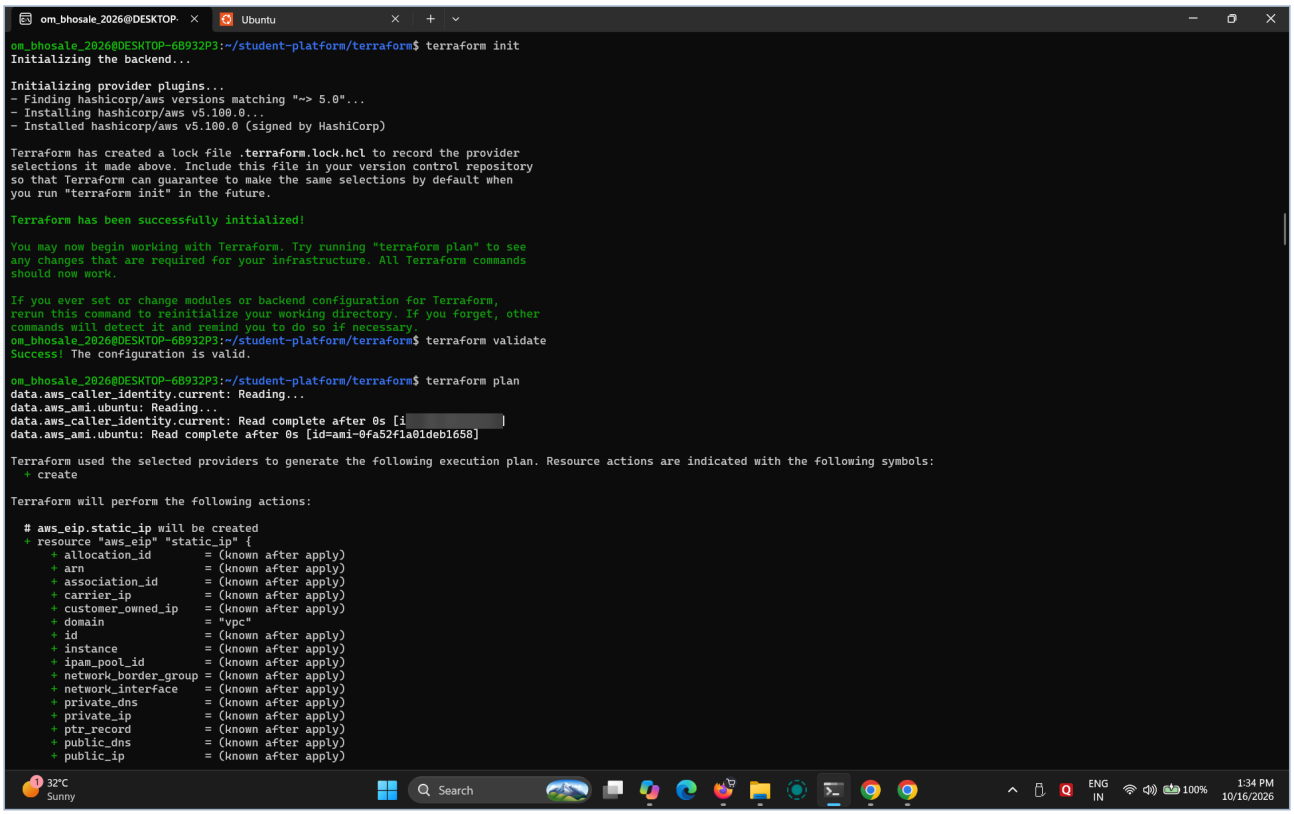


Figure 12: terraform init, validate and the start of the plan (account ID blurred)

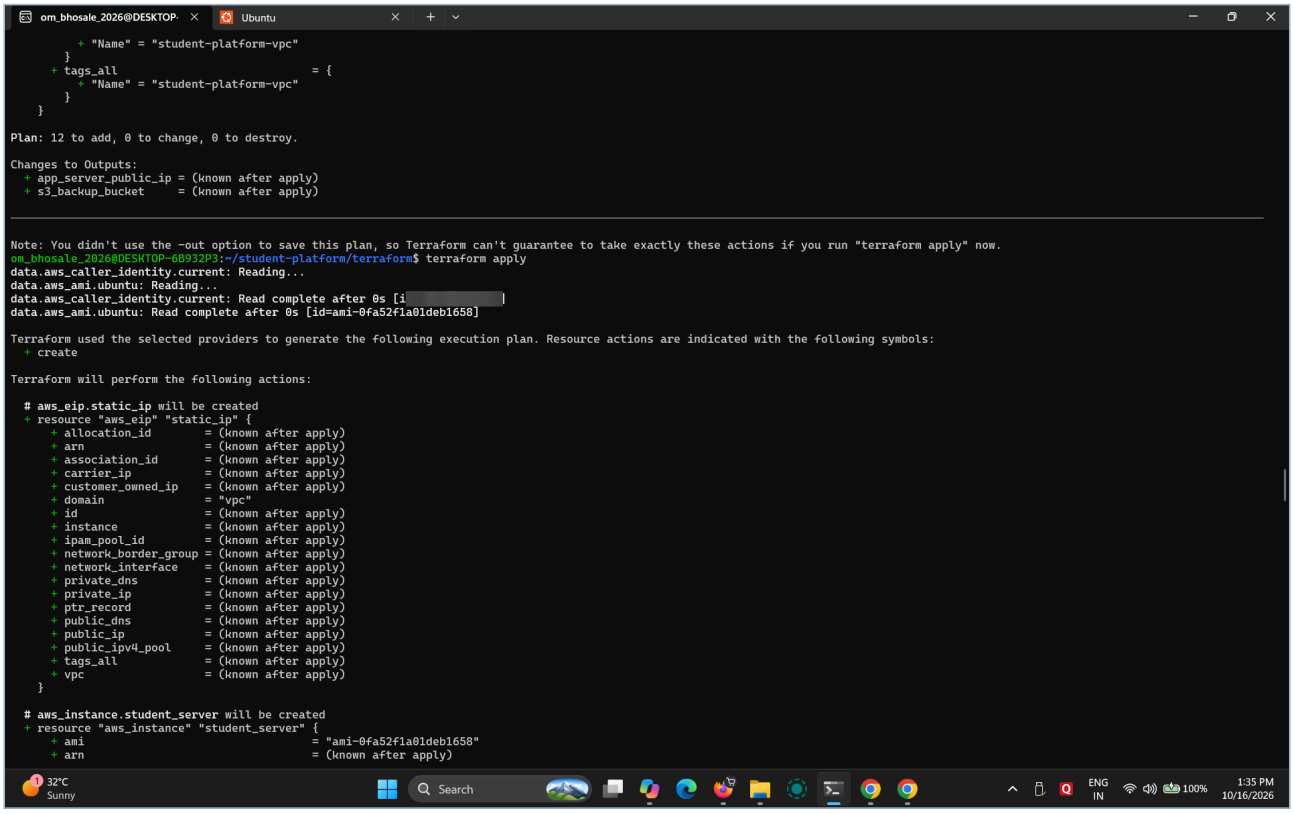


Figure 13: plan summary (12 to add) and terraform apply

The apply created the VPC, networking, security group, S3 bucket, EC2 instance and Elastic IP. That Elastic IP became the Ansible target in the next section.

8. Ansible: configuring the server

Ansible takes the empty Ubuntu instance and turns it into a working application host: base packages and firewall, Docker, Nginx, then the Student Record container. The project is under `~/student-platform/ansible` and is split into four roles so each concern stays separate.

```
mkdir -p ~/student-platform/ansible/{inventory,group_vars,roles/{common,docker,nginx,student_app}/tasks,roles/nginx/t
```

Role	Tasks
common	Update the APT cache, install curl, git, jq, ufw and python3-pip, allow SSH (22) and HTTP (80) through UFW, enable UFW
docker	Install prerequisites, add Docker's GPG key and official repository, install Docker Engine with the CLI, containerd, buildx and compose plugins, start and enable the service, add the ubuntu user to the docker group
nginx	Install Nginx, render the reverse-proxy template `nginx.conf.j2` into sites-available, enable it with a symlink, remove the default site, restart Nginx through a handler
student_app	Create the persistent data directory for SQLite, pull and start the Student Record container, wait for the app's health endpoint

The playbook `playbook.yml` targets the host group `student_servers` with privilege escalation and runs the roles in the order common, docker, nginx, student_app. Shared values (app_port 8080, app_image, server_name) sit in `group_vars/student_servers.yml`, so the tasks stay identical when only the values change.

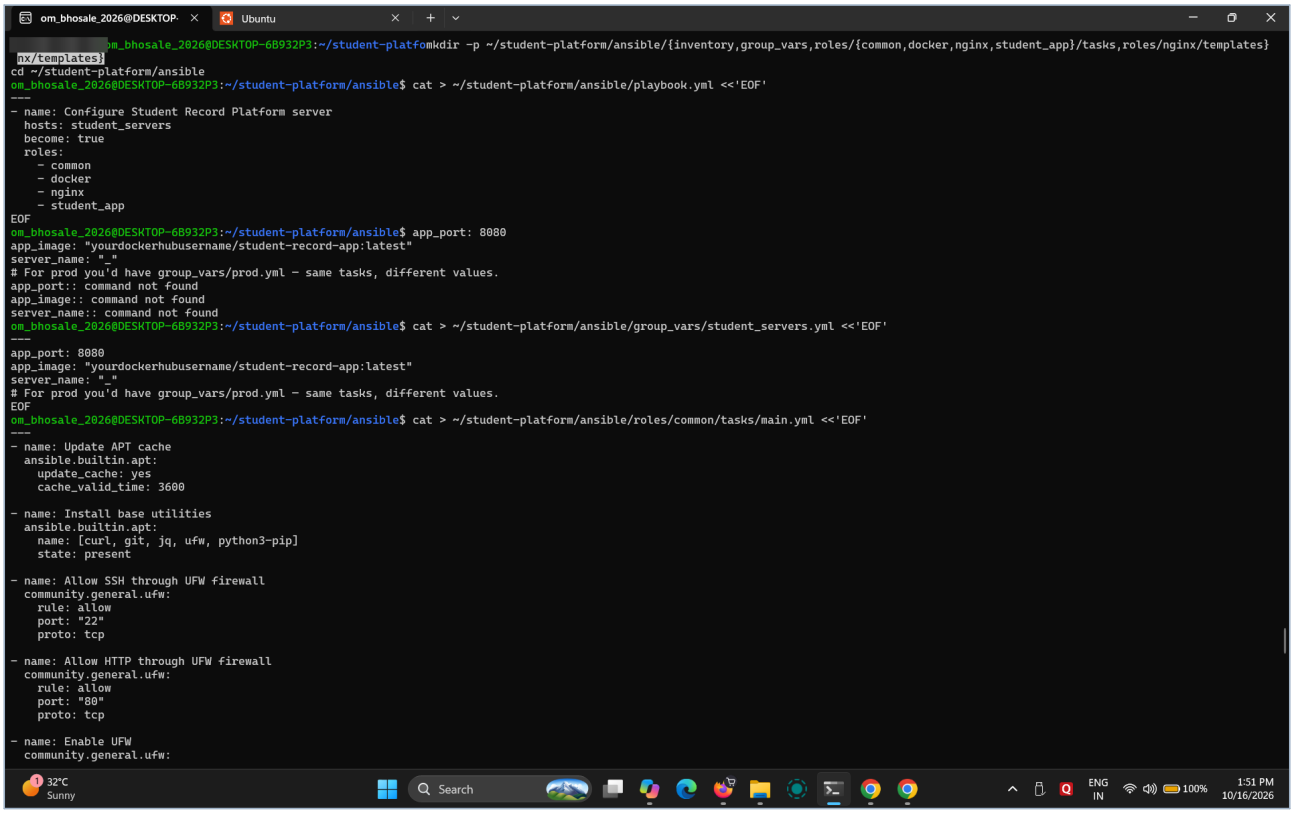


Figure 14: playbook.yml, group variables and the start of the common role (the first paste of the variables at the prompt failed, then the file was created with cat)

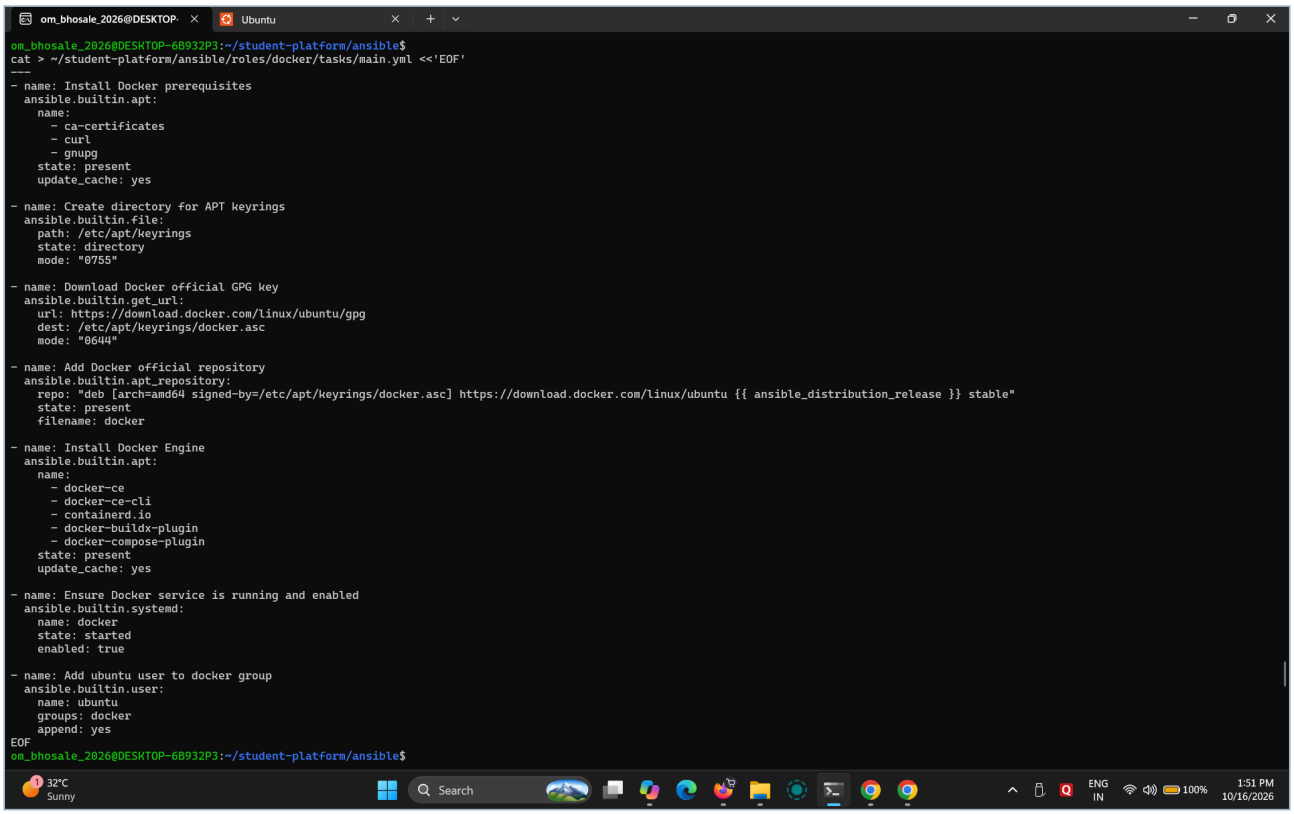


Figure 15: docker role tasks

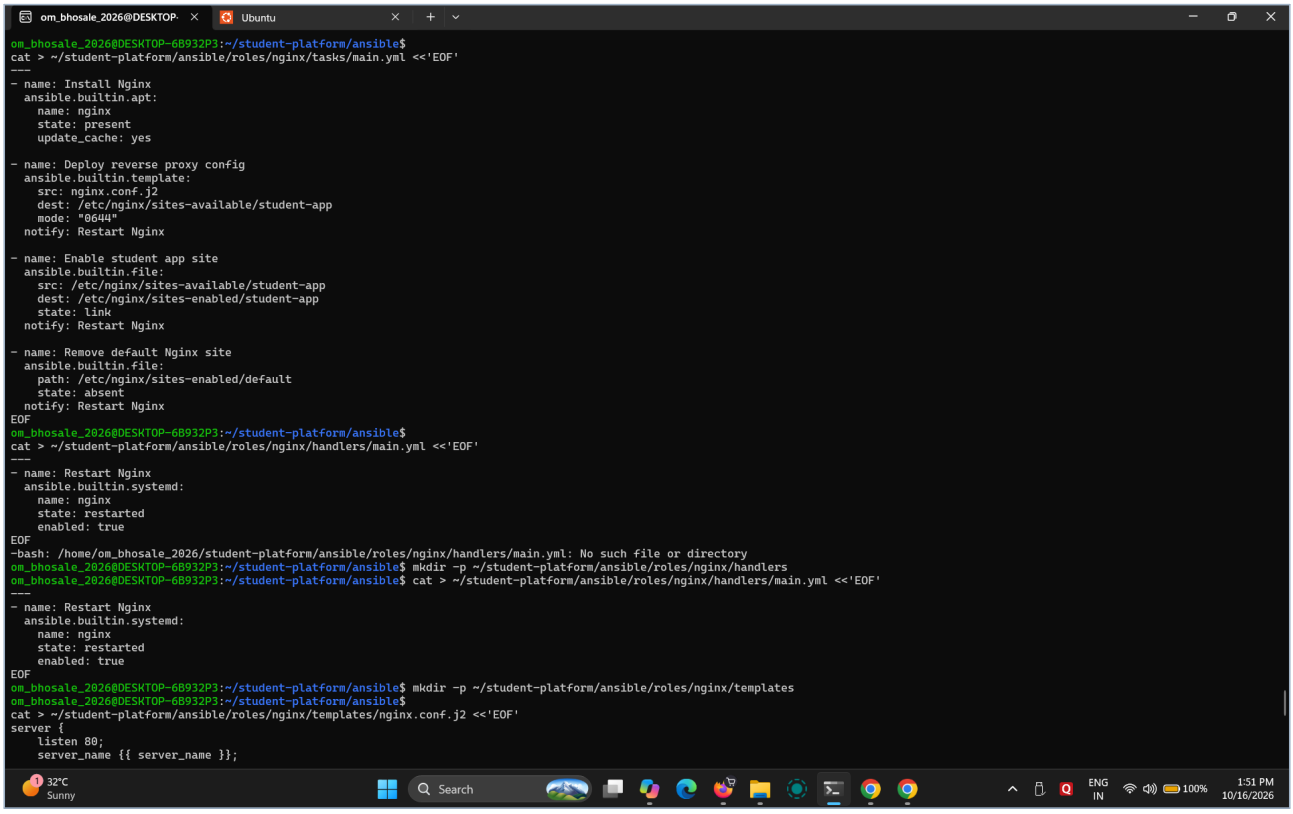


Figure 16: nginx role tasks, restart handler and the start of the Nginx template

The helper script `deploy.sh` ties Terraform and Ansible together. It validates the Terraform configuration, reads the server address with `terraform output -raw app_server_public_ip`, writes `inventory/hosts.ini` (user ubuntu, private key student-aws-key), installs the Ansible collections and runs a syntax check. It deliberately stops before deploying so the region, state, inventory IP and Docker image can be reviewed first.

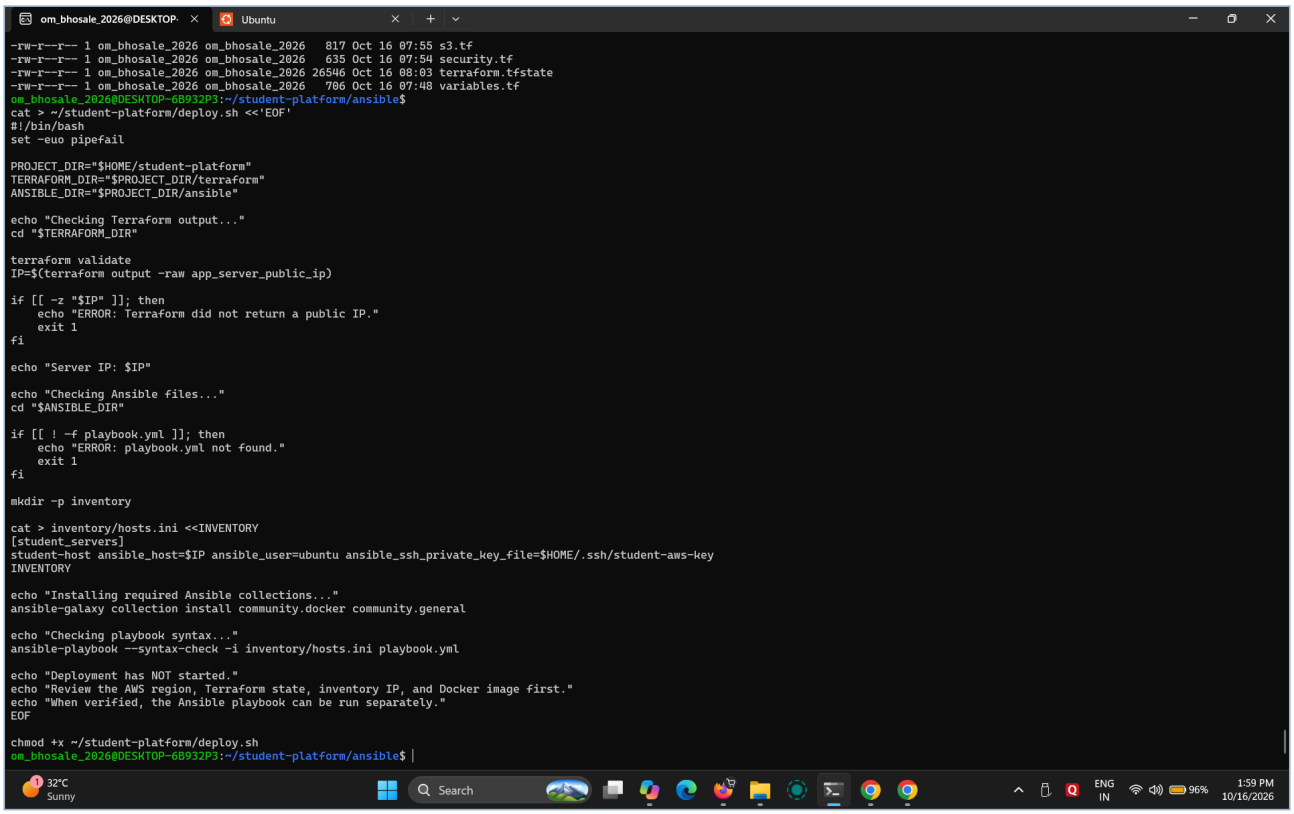


Figure 17: deploy.sh pre-flight script

9. Deployment and results

The playbook ran to completion with 0 failed and 0 unreachable hosts, and the application's health check passed. First, `deploy.sh` validated Terraform, printed the server IP (blurred), confirmed the collections were installed and ran the syntax check, which accepted `playbook.yml`. The syntax check also printed a deprecation notice: `ansible.builtin.apt_repository` is replaced by `deb822_repository` and will be removed in ansible-core 2.25. It is informational and does not affect this run.

```
~/student-platform/deploy.sh
ansible-playbook -i inventory/hosts.ini playbook.yml
```

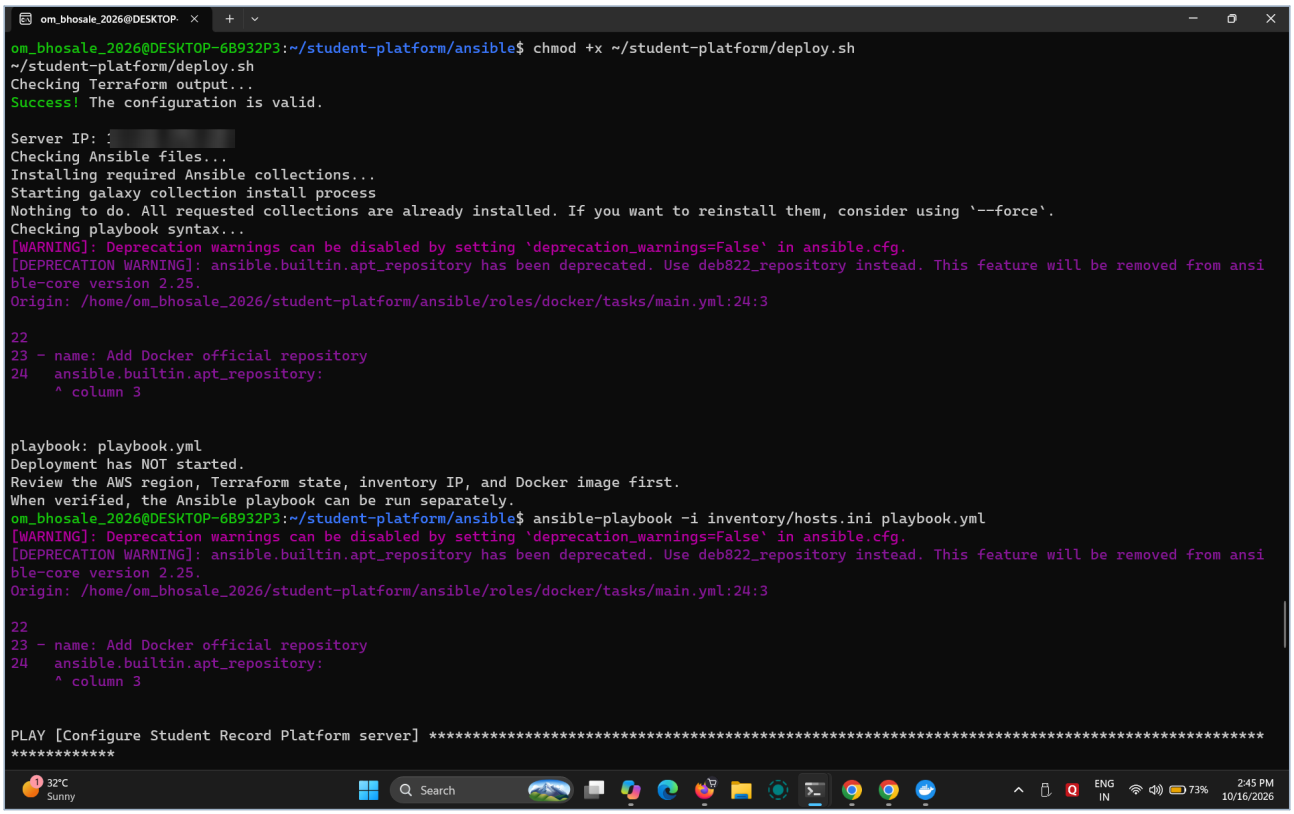


Figure 18: deploy.sh output, syntax check and the start of the playbook run (server IP blurred)

The run recap in Figure 19 shows each stage of the server build.

Stage	Result
Docker Engine installed	changed
Docker service running and enabled	ok
ubuntu user added to docker group	changed
Nginx installed, proxy config deployed, site enabled, default site removed	changed
SQLite data directory created	changed
Student Record container pulled and launched	changed
App health endpoint wait	ok
Nginx restart handler	changed

Play recap for student-host: ok=21, changed=16, unreachable=0, failed=0, skipped=0, rescued=0, ignored=0.

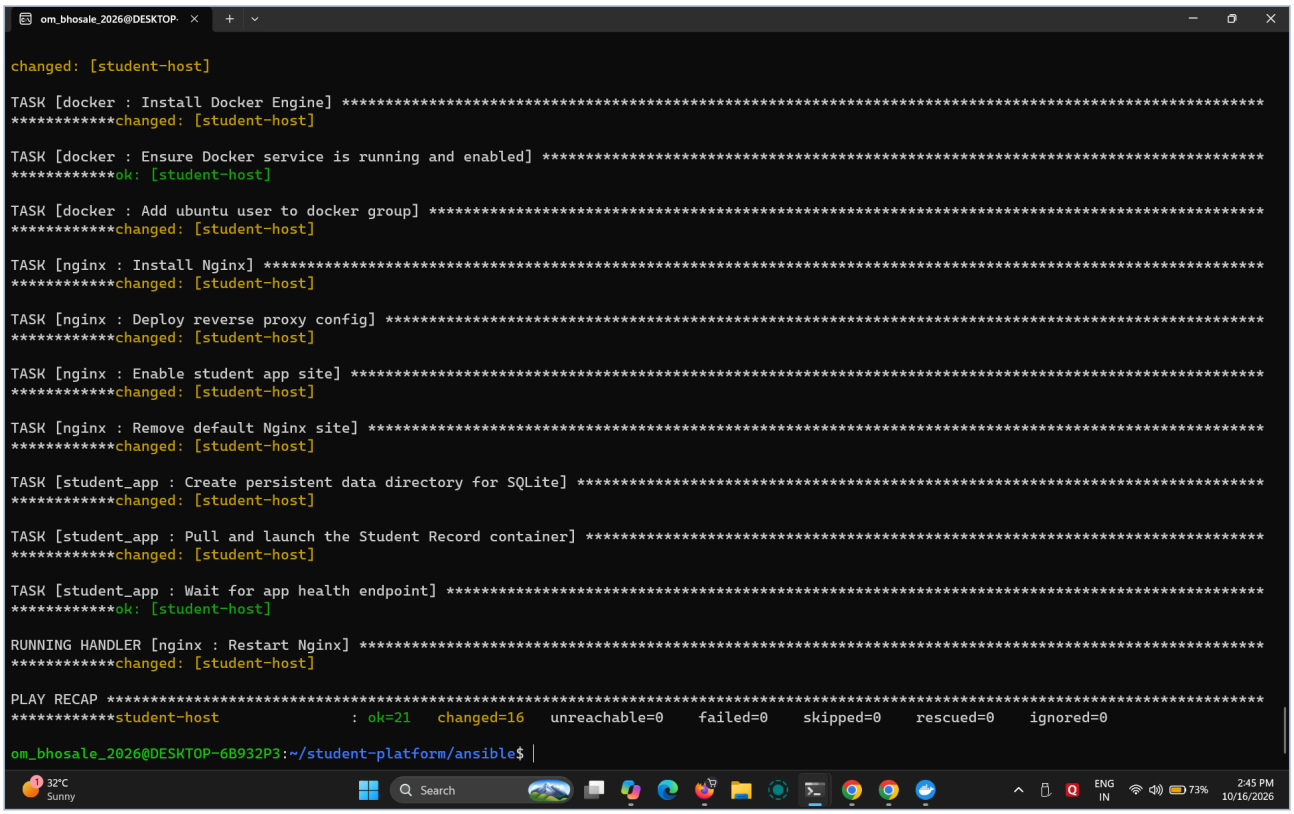


Figure 19: Ansible task results and PLAY RECAP (ok=21, changed=16, failed=0)

10. Cleanup: destroying the infrastructure

After the deployment was verified, everything was removed so no AWS charges continue. `terraform destroy` finished with "Destroy complete! Resources: 12 destroyed", matching the 12 resources the plan created. The checks for optional remote-state resources returned NoSuchBucket and ResourceNotFoundException, which confirms the S3 backend and the terraform-locks table were never created. The key pair student-aws-key was deleted from us-east-1 ("Return": true), and the local project folder was removed after the destroy had completed.

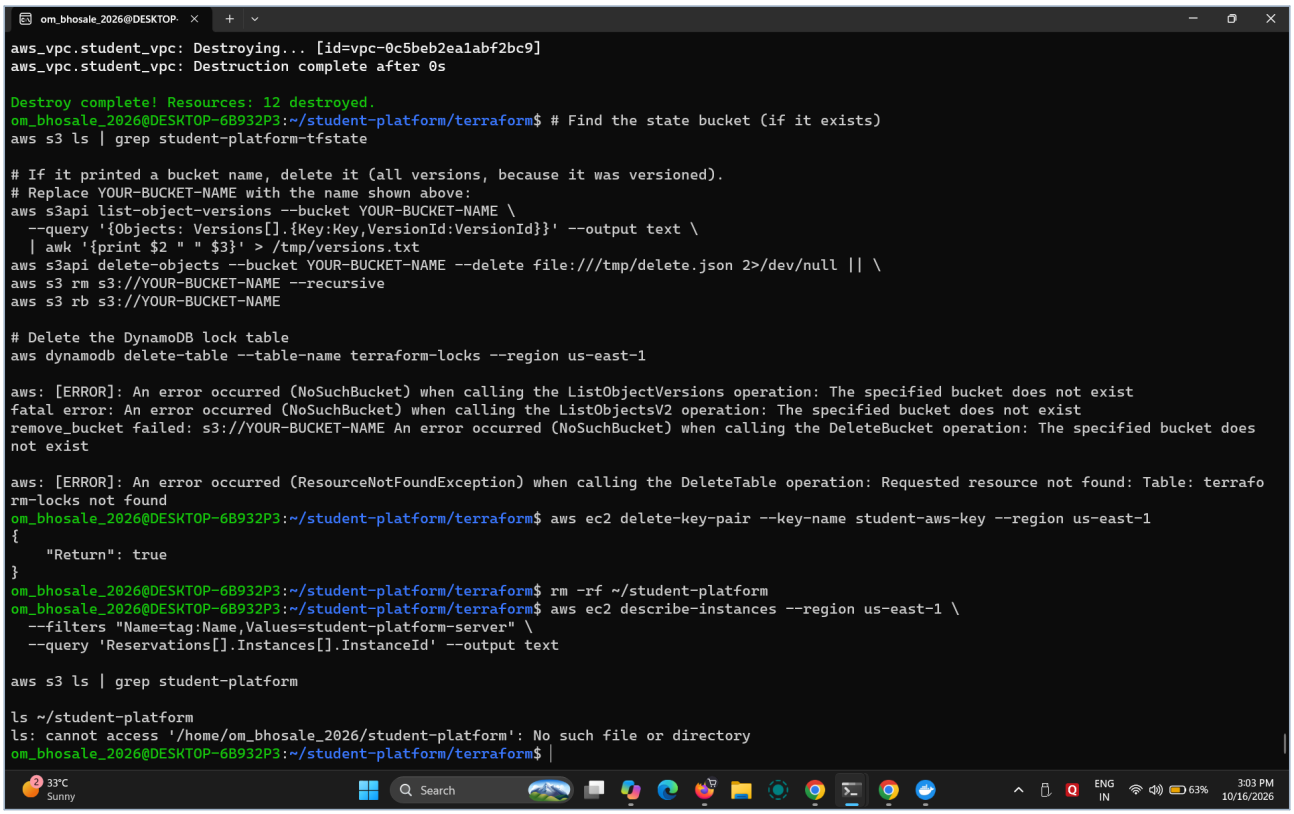


Figure 20: terraform destroy (12 destroyed), remote-state checks, key pair deletion and local cleanup

The infrastructure lived in ap-south-1, while the instance query in Figure 20 used us-east-1. These region-correct checks complete the cleanup, and the first three should print nothing; the last command removes the key pair from that region if it was imported there:

```
aws ec2 describe-instances --region ap-south-1 \
  --filters "Name=tag:Name,Values=student-platform-server" "Name=instance-state-name,Values=pending,running,stopping,
  --query 'Reservations[].Instances[].InstanceId' --output text
aws ec2 describe-addresses --region ap-south-1 --query 'Addresses[].PublicIp' --output text
aws s3 ls | grep student-platform
aws ec2 delete-key-pair --key-name student-aws-key --region ap-south-1
```

11. Issues encountered and resolutions

Nine problems came up during the lab; none blocked the final result.

Problem	Cause	Resolution
Errors such as `terraform has no command named "{"` and `command not found` after pasting code	The shell ran each pasted line as a command	Save code to a file with `cat > file <<'EOF' ... EOF`, check it with `cat`, then run the tool (Figure 14)
`No such file or directory` when writing `handlers/main.yml`	The handlers folder did not exist yet	`mkdir -p roles/nginx/handlers`, then write the file again (Figure 16)
gpg "File exists" and "no valid OpenPGP data found" for the HashiCorp key	The keyring file was already present from an earlier install	The existing key still verified the repository; add `--yes` to `gpg --dearmor` to refresh it (Figure 4)
`terraform -install-autocomplete` reported an error	Autocomplete was already installed in `~/.bashrc`	No action needed (Figure 4)
`aws configure` showed "Stopped"	The command was suspended with Ctrl+Z before it saved	Ran `aws configure` again to completion (Figure 6)
ssh-keygen overwrite failed with Permission denied	The existing private key is read-only (mode 400)	Kept the existing key pair and reapplied permissions (Figure 7)
Warning that `~/.aws/credentials` is accessible by other users	Default file permissions were too open	`chmod 600 ~/.aws/credentials` (section 4)

Problem	Cause	Resolution
`apt_repository` deprecation warning from Ansible	The module is superseded by `deb822_repository`	Informational; no effect on the run (Figure 18)
Key pair created in us-east-1 but the instance launches in ap-south-1	Key pairs are regional, and the lab guide uses us-east-1	Register the same public key in the deployment region (section 5)

12. Security governance and conclusion

The lab met its security requirements:

- Credentials live only in `~/.aws/`, never in Git, `.tf` files or playbooks; keys, account ID and IP addresses are blurred in every screenshot.
- The SSH private key is mode 400, and the security group allows SSH from one administrator address only.
- The security group exposes only ports 22 and 80, with the application reached through Nginx.
- The S3 backup bucket blocks all public access, and the EC2 root volume is encrypted.
- All resources were destroyed after verification, so no charges continue.

Chapter 2 is complete: AWS CLI 2.38.1, Terraform 1.16.6 and Ansible core 2.21.5 are installed and verified, the CLI authenticates as devops-student, and the SSH key pair is created and imported. Building on that, Terraform provisioned 12 AWS resources and Ansible configured the server with Docker, Nginx and the Student Record container with 0 failed tasks.